

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт Перспективной инженерии
Департамент цифровых и робототехнических систем и электроники

ОТЧЕТ
ПО ПРАКТИЧЕСКОЙ РАБОТЕ № 5
дисциплины «Искусственный Интеллект и Машинное Обучение»

Выполнил: Мендеш Пашкоал Педру
2 курс, группа ИВТ-б-о-24-1,
09.03.01 «Информатика и Вычислительная
техника», направленность (профиль)
«Программное обеспечение средств
вычислительной техники и
автоматизированных систем», очная форма
обучения

(подпись)

Руководитель практики:
Воронкин Роман А. доцент факультета
цифровых, робототехнических систем и
электроники института перспективной
инженерии.

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2026 г.

Тема: Введение в pandas: изучение структуры Series и базовых операций

Цель работы: познакомить с основами работы с библиотекой pandas, в частности, со структурой данных Series.

Этапы выполнения лабораторных работ

1. Изучение теоретического материала
2. Создание репозитория на GitHub:

<https://github.com/Pascoalpm/pandas-dataframe-lab.git>

Comentado [PM1]: <https://github.com/Pascoalpm/pandas-dataframe-lab.git>

3. Клонирование репозитория:
4. Настройка .gitignore для Python, Jupyter.
5. Запуск JupyterLab:
6. Выполнение заданий

Примеры практической работы

- **Создание Series**

```
import pandas as pd

s = pd.Series([10, 20, 30, 40], index=["a", "b", "c", "d"])
print(s)
```

a	10
b	20
c	30
d	40

dtype: int64

Рисунок 1. Пример 1

- **Создание DataFrame из словаря списков**

```
import pandas as pd

data = {
    "Имя": ["Анна", "Иван", "Ольга"],
    "Возраст": [25, 30, 22],
    "Город": ["Москва", "Санкт-Петербург", "Казань"]
}
df = pd.DataFrame(data)
print(df)
```

	Имя	Возраст	Город
0	Анна	25	Москва
1	Иван	30	Санкт-Петербург
2	Ольга	22	Казань

Рисунок 2. Пример 2

- **Создание DataFrame из словаря Series**

```
import pandas as pd

ages = pd.Series([25, 30, 22], index=["a", "b", "c"])
cities = pd.Series(["Москва", "СПб", "Казань"], index=["a", "b", "c"])

df = pd.DataFrame({"Возраст": ages, "Город": cities})
print(df)
```

	Возраст	Город
a	25	Москва
b	30	СПб
c	22	Казань

Рисунок 3. Пример 3

- **Создание DataFrame из списка списков**

```
import pandas as pd

data = [
    ["Анна", 25, "Москва"],
    ["Иван", 30, "СПб"],
    ["Ольга", 22, "Казань"]
]

df = pd.DataFrame(data, columns=["Имя", "Возраст", "Город"])
print(df)
```

	Имя	Возраст	Город
0	Анна	25	Москва
1	Иван	30	СПб
2	Ольга	22	Казань

Рисунок 4. Пример 4

- **Создание DataFrame из списка словарей (с пропусками NaN)**

```
import pandas as pd

data = [
    {"Имя": "Анна", "Возраст": 25, "Город": "Москва"},
    {"Имя": "Иван", "Возраст": 30},
    {"Имя": "Ольга", "Возраст": 22, "Город": "Казань"}
]

df = pd.DataFrame(data)
print(df)
```

	Имя	Возраст	Город
0	Анна	25	Москва
1	Иван	30	NaN
2	Ольга	22	Казань

Рисунок 5. Пример 5

- **Создание DataFrame из массива NumPy**

```
import pandas as pd
import numpy as np

data = np.array([
    ["Анна", 25, "Москва"],
    ["Иван", 30, "СПб"],
    ["Ольга", 22, "Казань"]
])

df = pd.DataFrame(data, columns=["Имя", "Возраст", "Город"])
print(df)
```

	Имя	Возраст	Город
0	Анна	25	Москва
1	Иван	30	СПб
2	Ольга	22	Казань

Рисунок 6. Пример 6

- **Создание пустого DataFrame и добавление строк**

```
import pandas as pd

df = pd.DataFrame(columns=['Имя', 'Возраст', 'Город'])
print("Пустой DataFrame:")
print(df)

df.loc[0] = ['Анна', 25, 'Москва']
df.loc[1] = ['Иван', 26, 'СПб']
print("\nDataFrame после добавления строк:")
print(df)
```

Пустой DataFrame:
Empty DataFrame
Columns: [Имя, Возраст, Город]
Index: []

DataFrame после добавления строк:

	Имя	Возраст	Город
0	Анна	25	Москва
1	Иван	26	СПб

Рисунок 7. Пример 7

- **Чтение данных из CSV**

```
import pandas as pd

# Создаём и сохраняем CSV файл
data = {
    "Имя": ["Анна", "Иван", "Ольга"],
    "Возраст": [25, 30, 22],
    "Город": ["Москва", "СПб", "Казань"]
}
df = pd.DataFrame(data)
df.to_csv("data.csv", index=False)

# Читаем CSV файл
df_read = pd.read_csv("data.csv")
print(df_read.head())
```

	Имя	Возраст	Город
0	Анна	25	Москва
1	Иван	30	СПб
2	Ольга	22	Казань

Рисунок 8. Пример 8

- **Чтение данных из Excel**

```
import pandas as pd

data = {
    "Имя": ["Анна", "Иван", "Ольга"],
    "Возраст": [25, 30, 22],
    "Город": ["Москва", "СПб", "Казань"]
}

df = pd.DataFrame(data)
print("DataFrame criado:")
print(df)

# Сохраняем в Excel
df.to_excel("data.xlsx", sheet_name="Лист1", index=False)
print("\nФайл Excel сохранен как 'data.xlsx'")

# Читаем Excel файл
df_excel = pd.read_excel("data.xlsx", sheet_name="Лист1")
print("\nДанные прочитаны из Excel файла:")
print(df_excel.head())
```

DataFrame criado:

	Имя	Возраст	Город
0	Анна	25	Москва
1	Иван	30	СПб
2	Ольга	22	Казань

Файл Excel сохранен как 'data.xlsx'

Данные прочитаны из Excel файла:

	Имя	Возраст	Город
0	Анна	25	Москва
1	Иван	30	СПб
2	Ольга	22	Казань

Рисунок 9. Пример 9

- Методы head() и tail()

```
import pandas as pd

data = {
    "Имя": ["Анна", "Иван", "Ольга", "Петр", "Мария", "Сергей"],
    "Возраст": [25, 30, 22, 40, 35, 28],
    "Город": ["Москва", "СПб", "Казань", "Новосибирск", "Екатеринбург", "Сочи"]
}
df = pd.DataFrame(data)

print("head(3):")
print(df.head(3))
print("\ntail(2):")
print(df.tail(2))

head(3):
   Имя  Возраст  Город
0  Анна      25  Москва
1  Иван      30   СПб
2  Ольга      22  Казань

tail(2):
   Имя  Возраст  Город
4  Мария      35  Екатеринбург
5  Сергей      28   Сочи
```

Рисунок 10. Пример 10

- Метод info()

```
import pandas as pd

data = {
    "Имя": ["Анна", "Иван", "Ольга"],
    "Возраст": [25, 30, 22],
    "Город": ["Москва", "СПб", "Казань"]
}
df = pd.DataFrame(data)
df.info()

<class 'pandas.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 3 columns):
#   Column  Non-Null Count  Dtype
---  ---
0   Имя      3 non-null      str
1   Возраст  3 non-null      int64
2   Город    3 non-null      str
dtypes: int64(1), str(2)
memory usage: 204.0 bytes
```

Рисунок 11. Пример 11

- Метод describe()

```
import pandas as pd

data = {
    "Возраст": [25, 30, 22, 40, 35, 28]
}
df = pd.DataFrame(data)
print(df.describe())

      Возраст
count    6.00000
mean     30.00000
std       6.60303
min      22.00000
25%      25.75000
50%      29.00000
75%      33.75000
max      40.00000
```

Рисунок 12. Пример 12

- Доступ к данным с .loc[]

```
import pandas as pd

data = {
    "Имя": ["Анна", "Иван", "Ольга", "Петр"],
    "Возраст": [25, 30, 22, 40],
    "Город": ["Москва", "СПб", "Казань", "Новосибирск"]
}
df = pd.DataFrame(data, index=[0, 1, 2, 3])

print("Строка с индексом 1:")
print(df.loc[1])
```

Строка с индексом 1:
Имя Иван
Возраст 30
Город СПб
Name: 1, dtype: object

Рисунок 13. Пример 13

- Доступ к данным с .iloc[]

```
import pandas as pd

data = {
    "Имя": ["Анна", "Иван", "Ольга", "Петр"],
    "Возраст": [25, 30, 22, 40],
    "Город": ["Москва", "СПб", "Казань", "Новосибирск"]
}
df = pd.DataFrame(data)

print("Третья строка (индекс 2):")
print(df.iloc[2])
```

Третья строка (индекс 2):
Имя Ольга
Возраст 22
Город Казань
Name: 2, dtype: object

Рисунок 14. Пример 14

Практические задания

1. Создание DataFrame разными способами

```
import pandas as pd
import numpy as np
from IPython.display import display

# 1.1 Из словаря список (первые 5 сотрудников из Таблицы 1)
data_dict = {
    "ID": [1, 2, 3, 4, 5],
    "Имя": ["Иван", "Ольга", "Алексей", "Мария", "Сергей"],
    "Возраст": [25, 30, 40, 35, 28],
    "Должность": ["Инженер", "Аналитик", "Менеджер", "Программист", "Специалист"],
    "Отдел": ["IT", "Маркетинг", "Продажи", "IT", "HR"],
    "Зарплата": [60000, 75000, 90000, 80000, 50000],
    "Стаж работы": [2, 5, 15, 7, 3]
}

df_dict = pd.DataFrame(data_dict)
print("DataFrame из словаря списков:")
display(df_dict)

# 1.2 Из списка словарей
data_list_dict = [
    {"ID": 1, "Имя": "Иван", "Возраст": 25, "Должность": "Инженер", "Отдел": "IT", "Зарплата": 60000, "Стаж работы": 2},
    {"ID": 2, "Имя": "Ольга", "Возраст": 30, "Должность": "Аналитик", "Отдел": "Маркетинг", "Зарплата": 75000, "Стаж работы": 5},
    {"ID": 3, "Имя": "Алексей", "Возраст": 40, "Должность": "Менеджер", "Отдел": "Продажи", "Зарплата": 90000, "Стаж работы": 15},
    {"ID": 4, "Имя": "Мария", "Возраст": 35, "Должность": "Программист", "Отдел": "IT", "Зарплата": 80000, "Стаж работы": 7},
    {"ID": 5, "Имя": "Сергей", "Возраст": 28, "Должность": "Специалист", "Отдел": "HR", "Зарплата": 50000, "Стаж работы": 3}
]

df_list_dict = pd.DataFrame(data_list_dict)
print("DataFrame из списка словарей:")
display(df_list_dict)

# 1.3 Из массива NumPy (случайные возрасты от 20 до 60)
np.random.seed(42)
ages = np.random.randint(20, 70, size=10)
df_numpy = pd.DataFrame(ages, columns=["Возраст"])
print("DataFrame из массива NumPy (случайные возрасты):")
display(df_numpy)

# 1.4 Проверка типов данных с .info()
print("Информация о DataFrame (типы данных):")
df_dict.info()
```

Рисунок 15. Код Задания 1

DataFrame из словаря списков:

	ID	Имя	Возраст	Должность	Отдел	Зарплата	Стаж работы
0	1	Иван	25	Инженер	IT	60000	2
1	2	Ольга	30	Аналитик	Маркетинг	75000	5
2	3	Алексей	40	Менеджер	Продажи	90000	15
3	4	Мария	35	Программист	IT	80000	7
4	5	Сергей	28	Специалист	HR	50000	3

DataFrame из списка словарей:

	ID	Имя	Возраст	Должность	Отдел	Зарплата	Стаж работы
0	1	Иван	25	Инженер	IT	60000	2
1	2	Ольга	30	Аналитик	Маркетинг	75000	5
2	3	Алексей	40	Менеджер	Продажи	90000	15
3	4	Мария	35	Программист	IT	80000	7
4	5	Сергей	28	Специалист	HR	50000	3

DataFrame из массива NumPy (случайные возраста):

	Возраст
0	66
1	56
2	42
3	35
4	48
5	66
6	46
7	50
8	38
9	38

Рисунок 16. ВЫПОЛНЕННЫЙ КОД

2. Чтение данных из файлов (CSV, Excel, JSON)

```
# Создаем полную Таблицу 2 (клиенты) для работы
data_clients = {
    "ID": range(1, 21),
    "Имя": ["Иван", "Ольга", "Алексей", "Мария", "Сергей", "Анна", "Дмитрий", "Елена", "Виктор", "Алиса",
            "Павел", "Светлана", "Роман", "Татьяна", "Николай", "Валерия", "Григорий", "Юлия", "Степан", "Васильева"],
    "Возраст": [34, 27, 45, 38, 29, 50, 31, 40, 28, 33, 46, 37, 41, 25, 39, 42, 49, 50, 30, 35],
    "Город": ["Москва", "Санкт-Петербург", "Казань", "Новосибирск", "Екатеринбург", "Воронеж", "Челябинск",
            "Краснодар", "Ростов-на-Дону", "Уфа", "Омск", "Пермь", "Тюмень", "Саратов", "Самара",
            "Волгоград", "Барнаул", "Иркутск", "Хабаровск", "Томск"],
    "Баланс на счете": [120000, 80000, 150000, 200000, 95000, 300000, 140000, 175000, 110000, 98000,
                       250000, 210000, 135000, 155000, 125000, 180000, 275000, 320000, 105000, 90000],
    "Кредитная история": ["Хорошая", "Средняя", "Плохая", "Хорошая", "Средняя", "Отличная", "Средняя",
                           "Хорошая", "Плохая", "Средняя", "Хорошая", "Отличная", "Средняя", "Хорошая",
                           "Средняя", "Плохая", "Отличная", "Хорошая", "Средняя", "Плохая"]
}
df_clients = pd.DataFrame(data_clients)

# 2.1 Сохраняем Таблицу 1 в CSV и читаем обратно
df_dict.to_csv("tabel1.csv", index=False)
df_csv = pd.read_csv("tabel1.csv")
print("CSV файл загружен, первые 5 строк:")
display(df_csv.head())

# 2.2 Сохраняем Таблицу 2 в Excel и читаем обратно
df_clients.to_excel("dados.xlsx", sheet_name="Клиенты", index=False)
df_excel = pd.read_excel("dados.xlsx", sheet_name="Клиенты")
print("\nExcel файл загружен, первые 5 строк:")
display(df_excel.head())

# 2.3 Сохраняем Таблицу 1 в JSON и читаем обратно
df_dict.to_json("tabel1.json", orient="records", indent=4)
df_json = pd.read_json("tabel1.json")
print("\nJSON файл загружен, первые 5 строк:")
display(df_json.head())
```

Рисунок 17. Код Задания 2

CSV файл загружен, первые 5 строк:

	ID	Имя	Возраст	Должность	Отдел	Зарплата	Стаж работы
0	1	Иван	25	Инженер	IT	60000	2
1	2	Ольга	30	Аналитик	Маркетинг	75000	5
2	3	Алексей	40	Менеджер	Продажи	90000	15
3	4	Мария	35	Программист	IT	80000	7
4	5	Сергей	28	Специалист	HR	50000	3

Рисунок 18. выполненный код

3. Доступ к данным (.loc, .iloc, .at, .iat)

```
# Используем Таблицу 1 с индексом по ID
df = df_dict.copy()
df.set_index("ID", inplace=True)

print("\n1. Информация о сотруднике с ID = 5 (используя .loc):")
print(df.loc[5])

print("\n2. Возраст третьего сотрудника в таблице (используя .iloc):")
print(f"Возраст: {df.iloc[2, df.columns.get_loc('Возраст')]}")

print("\n3. Название отдела для сотрудника 'Мария' (используя .at):")
# Находим ID Марии
maria_id = df[df["Имя"] == "Мария"].index[0]
print(f"Отдел: {df.at[maria_id, 'Отдел']}")

print("\n4. Зарплата сотрудника в 4-й строке и 5-м столбце (используя .iat):")
print(f"Зарплата: {df.iat[3, 4]}")

print("\n★ Разница между методами:")
print(".loc[] - доступ по меткам (названиям индексов и столбцов)")
print(".iloc[] - доступ по числовым позициям (номерам строк и столбцов)")
print(".at[] - быстрый доступ к одному элементу по меткам")
print(".iat[] - быстрый доступ к одному элементу по числовым позициям")
```

Рисунок 19. Код Задания 3

```
1. Информация о сотруднике с ID = 5 (используя .loc):
Имя          Сергей
Возраст      28
Должность     Специалист
Отдел         HR
Зарплата      50000
Стаж работы   3
Name: 5, dtype: object

2. Возраст третьего сотрудника в таблице (используя .iloc):
Возраст: 40

3. Название отдела для сотрудника 'Мария' (используя .at):
Отдел: IT

4. Зарплата сотрудника в 4-й строке и 5-м столбце (используя .iat):
Зарплата: 80000

★ Разница между методами:
.loc[] - доступ по меткам (названиям индексов и столбцов)
.iloc[] - доступ по числовым позициям (номерам строк и столбцов)
.at[] - быстрый доступ к одному элементу по меткам
.iat[] - быстрый доступ к одному элементу по числовым позициям
```

Рисунок 20. выполненный код

4. Добавление новых столбцов и строк

```
df = df_dict.copy()

# 4.1 Добавляем столбец "Категория зарплаты"
def salary_category(salary):
    if salary < 60000:
        return "Низкая"
    elif salary < 100000:
        return "Средняя"
    else:
        return "Высокая"

df["Категория зарплаты"] = df["Зарплата"].apply(salary_category)
print("DataFrame с новым столбцом 'Категория зарплаты':")
display(df)

# 4.2 Добавляем одного нового сотрудника через .loc
df.loc[21] = ["Антон", 32, "Разработчик", "IT", 85000, 6, "Средняя"]
print("\nDataFrame после добавления сотрудника Антон:")
display(df.tail(3))

# 4.3 Добавляем двух новых сотрудников через pd.concat()
new_employees = pd.DataFrame([
    [22, "Екатерина", 29, "Аналитик", "Маркетинг", 72000, 4, "Средняя"],
    [23, "Максим", 41, "Руководитель", "Продажи", 120000, 12, "Высокая"]
], columns=df.columns)

df = pd.concat([df, new_employees], ignore_index=True)
print("\nDataFrame после добавления двух сотрудников (Екатерина и Максим):")
display(df.tail(3))
```

DataFrame с новым столбцом 'Категория зарплаты':

	ID	Имя	Возраст	Должность	Отдел	Зарплата	Стаж работы	Категория зарплаты
0	1	Иван	25	Инженер	IT	60000	2	Средняя
1	2	Ольга	30	Аналитик	Маркетинг	75000	5	Средняя
2	3	Алексей	40	Менеджер	Продажи	90000	15	Средняя
3	4	Мария	35	Программист	IT	80000	7	Средняя
4	5	Сергей	28	Специалист	HR	50000	3	Низкая

Рисунок 21. Код Задания 4

5. Удаление строк и столбцов

```
df = df_dict.copy()
df["Категория зарплаты"] = df["Зарплата"].apply(salary_category)

# 5.1 Удаляем столбец "Категория зарплаты" с .drop()
df = df.drop(columns=["Категория зарплаты"])
print("После удаления столбца 'Категория зарплаты':")
display(df.head())

# 5.2 Удаляем строку с ID = 10
df = df[df["ID"] != 10]
print("\nПосле удаления строки с ID=10:")
display(df)

# 5.3 Удаляем строки, где Стаж работы < 3 лет
df = df[df["Стаж работы"] >= 3]
print("\nПосле удаления сотрудников со стажем < 3 лет:")
display(df)

# 5.4 Оставляем только столбцы: Имя, Должность, Зарплата
df = df[["Имя", "Должность", "Зарплата"]]
print("\nПосле удаления всех столбцов, кроме Имя, Должность, Зарплата:")
display(df)
```

Рисунок 22. Код Задания 4

После удаления столбца 'Категория зарплаты':

	ID	Имя	Возраст	Должность	Отдел	Зарплата	Стаж работы
0	1	Иван	25	Инженер	IT	60000	2
1	2	Ольга	30	Аналитик	Маркетинг	75000	5
2	3	Алексей	40	Менеджер	Продажи	90000	15
3	4	Мария	35	Программист	IT	80000	7
4	5	Сергей	28	Специалист	HR	50000	3

После удаления строки с ID=10:

	ID	Имя	Возраст	Должность	Отдел	Зарплата	Стаж работы
0	1	Иван	25	Инженер	IT	60000	2
1	2	Ольга	30	Аналитик	Маркетинг	75000	5
2	3	Алексей	40	Менеджер	Продажи	90000	15
3	4	Мария	35	Программист	IT	80000	7
4	5	Сергей	28	Специалист	HR	50000	3

После удаления сотрудников со стажем < 3 лет:

	ID	Имя	Возраст	Должность	Отдел	Зарплата	Стаж работы
1	2	Ольга	30	Аналитик	Маркетинг	75000	5
2	3	Алексей	40	Менеджер	Продажи	90000	15
3	4	Мария	35	Программист	IT	80000	7
4	5	Сергей	28	Специалист	HR	50000	3

После удаления всех столбцов, кроме Имя, Должность, Зарплата:

	Имя	Должность	Зарплата
1	Ольга	Аналитик	75000
2	Алексей	Менеджер	90000
3	Мария	Программист	80000
4	Сергей	Специалист	50000

Рисунок 23. выполненный код

6. Фильтрация данных (query, isin, between)

```
df = df_clients.copy()

# 6.1 Фильтрация через .isin() - клиенты из Москвы или Санкт-Петербурга
filtered_isin = df[df["Город"].isin(["Москва", "Санкт-Петербург"])]
print("\n1. Клиенты из Москвы или Санкт-Петербурга:")
display(filtered_isin)

# 6.2 Фильтрация через .between() - баланс от 100 000 до 250 000
filtered_between = df[df["Баланс на счете"].between(100000, 250000)]
print("\n2. Клиенты с балансом от 100 000 до 250 000:")
display(filtered_between)

# 6.3 Фильтрация через .query() - кредитная история "Хорошая" и баланс > 150 000
filtered_query = df.query("`Кредитная история` == 'Хорошая' and `Баланс на счете` > 150000")
print("\n3. Клиенты с кредитной историей 'Хорошая' и балансом > 150 000:")
display(filtered_query)
```

Рисунок 24. Код Задания 6

1. Клиенты из Москвы или Санкт-Петербурга:						
	ID	Имя	Возраст	Город	Баланс на счете	Кредитная история
0	1	Иван	34	Москва	120000	Хорошая
1	2	Ольга	27	Санкт-Петербург	80000	Средняя
2. Клиенты с балансом от 100 000 до 250 000:						
	ID	Имя	Возраст	Город	Баланс на счете	Кредитная история
0	1	Иван	34	Москва	120000	Хорошая
2	3	Алексей	45	Казань	150000	Плохая
3	4	Мария	38	Новосибирск	200000	Хорошая
6	7	Дмитрий	31	Челябинск	140000	Средняя
7	8	Елена	40	Краснодар	175000	Хорошая
8	9	Виктор	28	Ростов-на-Дону	110000	Плохая
10	11	Павел	46	Омск	250000	Хорошая
11	12	Светлана	37	Пермь	210000	Отличная
12	13	Роман	41	Тюмень	135000	Средняя
13	14	Татьяна	25	Саратов	155000	Хорошая
14	15	Николай	39	Самара	125000	Средняя
15	16	Валерия	42	Волгоград	180000	Плохая
18	19	Степан	30	Хабаровск	105000	Средняя
3. Клиенты с кредитной историей 'Хорошая' и балансом > 150 000:						
	ID	Имя	Возраст	Город	Баланс на счете	Кредитная история
3	4	Мария	38	Новосибирск	200000	Хорошая
7	8	Елена	40	Краснодар	175000	Хорошая
10	11	Павел	46	Омск	250000	Хорошая
13	14	Татьяна	25	Саратов	155000	Хорошая
17	18	Юлия	50	Иркутск	320000	Хорошая

Рисунок 25. выполненный код

7. Подсчет значений (count, value_counts, unique, nunique)

```
import pandas as pd

data_clients = {
    "ID": list(range(1, 21)),
    "Имя": ["Иван", "Ольга", "Алексей", "Мария", "Сергей", "Анна", "Дмитрий", "Елена",
            "Виктор", "Алиса", "Павел", "Светлана", "Роман", "Татьяна", "Николай",
            "Валерия", "Григорий", "Юлия", "Степан", "Васильева"],
    "Возраст": [34, 27, 45, 38, 29, 50, 31, 40, 28, 33, 46, 37, 41, 25, 39, 42, 49, 50, 30, 35],
    "Город": ["Москва", "Санкт-Петербург", "Казань", "Новосибирск", "Екатеринбург", "Воронеж",
            "Челябинск", "Краснодар", "Ростов-на-Дону", "Уфа", "Омск", "Пермь", "Тюмень",
            "Саратов", "Самара", "Волгоград", "Барнаул", "Иркутск", "Хабаровск", "Томск"],
    "Баланс на счете": [120000, 80000, 150000, 200000, 95000, 300000, 140000, 175000,
                       110000, 98000, 250000, 210000, 135000, 155000, 125000, 180000,
                       275000, 320000, 105000, 90000],
    "Кредитная история": ["Хорошая", "Средняя", "Плохая", "Хорошая", "Средняя", "Отличная",
                           "Средняя", "Хорошая", "Плохая", "Средняя", "Хорошая", "Отличная",
                           "Средняя", "Хорошая", "Средняя", "Плохая", "Отличная", "Хорошая",
                           "Средняя", "Плохая"]
}

df_clients = pd.DataFrame(data_clients)

df = df_clients.copy()

print("1. Количество непустых значений в каждом столбце (.count()):")
print(df.count())

print("\n2. Частота встречаемости значений в столбце 'Город' (.value_counts()):")
print(df["Город"].value_counts())

print("\n3. Количество уникальных значений (.unique()):")
print(f"Уникальных городов: {df['Город'].unique()}")
print(f"Уникальных возрастов: {df['Возраст'].unique()}")
print(f"Уникальных балансов: {df['Баланс на счете'].unique()}")
```

Рисунок 26. Код Задания 7

```

1. Количество непустых значений в каждом столбце (.count()):
ID                20
Имя               20
Возраст           20
Город             20
Баланс на счете   20
Кредитная история 20
dtype: int64

2. Частота встречаемости значений в столбце 'Город' (.value_counts()):
Город
Москва                1
Санкт-Петербург       1
Казань                 1
Новосибирск           1
Екатеринбург          1
Воронеж               1
Челябинск             1
Краснодар             1
Ростов-на-Дону        1
Уфа                   1
Омск                  1
Пермь                 1
Тюмень               1
Саратов              1
Самара               1
Волгоград            1
Барнаул              1
Иркутск              1
Хабаровск            1
Томск                1
Name: count, dtype: int64

3. Количество уникальных значений (.unique()):
Уникальных городов: <StringArray>
[
    'Москва', 'Санкт-Петербург', 'Казань', 'Новосибирск',
    'Екатеринбург', 'Воронеж', 'Челябинск', 'Краснодар',
    'Ростов-на-Дону', 'Уфа', 'Омск', 'Пермь',
    'Тюмень', 'Саратов', 'Самара', 'Волгоград',
    'Барнаул', 'Иркутск', 'Хабаровск', 'Томск']
Length: 20, dtype: str
Уникальных возрастов: [34 27 45 38 29 50 31 40 28 33 46 37 41 25 39 42 49 30 35]
Уникальных балансов: [120000  80000 150000 200000  95000 300000 140000 175000 110000  90000
 250000 210000 135000 155000 125000 180000 275000 320000 105000  90000]

```

Рисунок 27. ВЫПОЛНЕННЫЙ КОД

8. Обнаружение пропусков (isna, notna)

```

import pandas as pd
from IPython.display import display

# Создаем Таблицу 3 с пропусками (ИСПРАВЛЕННАЯ)
data_nan = {
    "ID": list(range(1, 21)),
    "Имя": ["Иван", "Ольга", "Алексей", "Мария", "Сергей", None, "Дмитрий", "Елена",
            "Виктор", "Алиса", "Павел", None, "Роман", "Татьяна", "Николай", "Валерия",
            "Григорий", "Юлия", "Степан", "Васильева"],
    "Возраст": [34.0, 27.0, None, 38.0, 29.0, 50.0, 31.0, 40.0, None, 33.0,
                46.0, 37.0, 41.0, 25.0, None, 42.0, 49.0, None, 30.0, 35.0],
    "Город": ["Москва", "Санкт-Петербург", "Казань", None, "Екатеринбург", "Воронеж",
              None, "Краснодар", "Ростов-на-Дону", "Уфа", "Омск", "Пермь", None,
              "Саратов", "Самара", None, "Барнаул", "Иркутск", "Хабаровск", "Томск"],
    "Баланс на счете": [120000.0, None, 150000.0, 200000.0, None, 300000.0, 140000.0,
                        175000.0, 110000.0, None, 250000.0, 210000.0, 135000.0,
                        155000.0, 125000.0, None, 275000.0, 320000.0, 105000.0, 90000.0],
    "Кредитная история": ["Хорошая", "Средняя", "Плохая", "Хорошая", None, "Отличная",
                           "Средняя", "Хорошая", None, "Средняя", "Хорошая", "Отличная",
                           "Средняя", "Хорошая", "Средняя", "Плохая", "Отличная",
                           "Хорошая", "Средняя", "Плохая"]
}

df_nan = pd.DataFrame(data_nan)

print("1. Количество NaN в каждом столбце (.isna().sum()):")
print(df_nan.isna().sum())

print("\n2. Количество заполненных значений в каждом столбце (.notna().sum()):")
print(df_nan.notna().sum())

print("\n3. DataFrame только с строками без пропусков (.dropna()):")
df_clean = df_nan.dropna()
display(df_clean)

```

Рисунок 28. Код Задания 8

```
1. Количество NaN в каждом столбце (.isna().sum()):
ID          0
Имя         2
Возраст     4
Город       4
Баланс на счете  4
Кредитная история  2
dtype: int64

2. Количество заполненных значений в каждом столбце (.notna().sum()):
ID          20
Имя         18
Возраст     16
Город       16
Баланс на счете  16
Кредитная история  18
dtype: int64

3. DataFrame только с строками без пропусков (.dropna()):
```

	ID	Имя	Возраст	Город	Баланс на счете	Кредитная история
0	1	Иван	34.0	Москва	120000.0	Хорошая
7	8	Елена	40.0	Краснодар	175000.0	Хорошая
10	11	Павел	46.0	Омск	250000.0	Хорошая
13	14	Татьяна	25.0	Саратов	155000.0	Хорошая
16	17	Григорий	49.0	Барнаул	275000.0	Отличная
18	19	Степан	30.0	Хабаровск	105000.0	Средняя
19	20	Васильева	35.0	Томск	90000.0	Плохая

Рисунок 29. выполненный код

Индивидуальное задание: Товары и магазины

17. Использовать `DataFrame`, содержащий следующие колонки: название товара; название магазина, в котором продается товар; стоимость товара в руб. Написать программу, выполняющую следующие действия: ввод с клавиатуры данных и добавление строк в `DataFrame`; записи должны быть размещены в алфавитном порядке по названиям товаров; вывод на экран информации о товаре, название которого введено с клавиатуры; если таких товаров нет, выдать на дисплей соответствующее сообщение.

```
import pandas as pd
import os

data_file = "products.csv" # Меняем для CSV

# Загружаем существующий DataFrame или создаем новый
if os.path.exists(data_file):
    df = pd.read_csv(data_file, encoding="utf-8")
    print(f"Загружено {len(df)} записей из {data_file}")
else:
    df = pd.DataFrame(columns=["Название товара", "Название магазина", "Стоимость товара"])
    print("Создан новый пустой DataFrame")

# Интерактивное добавление товаров
print("\n--- Добавление новых товаров ---")
print("(Для выхода оставьте название товара пустым)")

while True:
    name = input("Название товара (Enter для выхода): ").strip()
    if not name:
        break
    shop = input("Название магазина: ").strip()
    try:
        price = float(input("Стоимость товара (руб): "))
    except ValueError:
        print("Некорректная цена! Товар не добавлен.")
        continue

    df.loc[len(df)] = [name, shop, price]
    print(f"Товар '{name}' добавлен!\n")

# Сортировка по алфавиту
if not df.empty:
    df = df.sort_values(by="Название товара").reset_index(drop=True)
else:
    print("\nНет данных для сортировки")

# Сохранение в CSV
df.to_csv(data_file, index=False, encoding="utf-8")
print(f"\nДанные сохранены в {data_file}")

# Вывод всех товаров
print("\n" + "-"*60)
print("СПИСОК ВСЕХ ТОВАРОВ (в алфавитном порядке):")
print("-"*60)
if not df.empty:
    for idx, row in df.iterrows():
        print(f"{idx+1}. {row['Название товара']} | {row['Название магазина']} | {row['Стоимость товара']:.2f} руб.")
else:
    print("Список товаров пуст")
print("-"*60)

# Поиск товара (интерактив)
print("\n--- ПОИСК ТОВАРА ---")
search_name = input("Введите название товара для поиска (Enter для пропуска): ").strip()

if search_name and not df.empty:
    result = df[df["Название товара"] == search_name]
    if result.empty:
        print(f"Товар '{search_name}' не найден.")
    else:
        print(f"\nИнформация о товаре '{search_name}':")
        print(result.to_string(index=False))
elif search_name and df.empty:
    print("Нет данных для поиска")
else:
    print("Поиск пропущен")
```

Рисунок 30. Код индивидуального задания

```

✔ Загружено 3 записей из products.csv

--- Добавление новых товаров ---
(Для выхода оставьте название товара пустым)
Название товара (Enter для выхода): Молоко
Название магазина: Пятёрочка
Стоимость товара (руб): 89.50
✔ Товар 'Молоко' добавлен!

Название товара (Enter для выхода): Хлеб
Название магазина: Магнит
Стоимость товара (руб): 45.00
✔ Товар 'Хлеб' добавлен!

Название товара (Enter для выхода): Яблоки
Название магазина: Ашан
Стоимость товара (руб): 120.00
✔ Товар 'Яблоки' добавлен!

Название товара (Enter для выхода): 
```

Рисунок 30. выполненный код

pandas-dataframe-lab / Lab 5 /  Add file ▾ ...

 Add files via upload 2429624 · 1 minute ago  History

Name	Last commit message	Last commit date
--		
 doc	Add files via upload	1 minute ago
 Задание	Add files via upload	1 minute ago
 Примеры	Add files via upload	1 minute ago
 README.md	Add files via upload	1 minute ago

README.md 

Студент: Мендеш Пашкоал Педру Группа: ИБТ-6-о-24-1 Вариант: 17

Лабораторная работа: pandas DataFrame

Работа с основными структурами данных библиотеки **pandas**:

- `Series` и `DataFrame`
- Создание из разных источников (`dict`, `list`, `NumPy`, `CSV`, `Excel`, `JSON`)

Рисунок 31. Лабораторные работы загружены в репозиторий Github.

Вывод

В ходе лабораторной работы были изучены основные принципы работы с библиотекой `pandas`: создание `DataFrame` из различных источников, доступ к данным с помощью `.loc[]`, `.iloc[]`, `.at[]`, `.iat[]`, добавление и удаление строк и столбцов, фильтрация данных с использованием `.query()`, `.isin()`, `.between()`, а также обработка пропущенных значений. Полученные знания применены на практике при решении индивидуального задания (вариант 17) по созданию программы для управления товарами с возможностью добавления, сортировки по алфавиту и поиска данных в `DataFrame`.

ОТВЕТЫ НА ВОПРОСЫ

1. Как создать `pandas.DataFrame` из словаря списков?

Используется словарь, где ключи — названия столбцов, а значения — списки данных. Все списки должны быть одинаковой длины.

2. В чем отличие создания `DataFrame` из списка словарей и словаря списков?

Словарь списков	Список словарей
Ключи → столбцы	Каждый словарь → строка
Все списки одинаковой длины	Словари могут иметь разные ключи
Удобен для создания таблицы целиком	Удобен для добавления строк по одной

При создании из словаря списков ключи становятся названиями столбцов. При создании из списка словарей каждый словарь представляет отдельную строку.

3. Как создать `pandas.DataFrame` из массива `NumPy`?

Используйте `pd.DataFrame()` напрямую, передав массив `NumPy` и указав названия столбцов. Все данные внутри массива должны быть одного типа.

4. Как загрузить `DataFrame` из CSV-файла, указав разделитель ;?

Используйте параметр `sep=";"` в функции `pd.read_csv()`.

5. Как загрузить данные из Excel в `pandas.DataFrame` и выбрать конкретный лист?

Используйте функцию `pd.read_excel()` с параметром `sheet_name`, указав название листа или его номер (0, 1, 2...).

6. Чем отличается чтение данных из JSON и Parquet в `pandas`?

JSON	Parquet
Текстовый формат	Бинарный формат
Медленнее для больших данных	Оптимизирован для больших данных

Читается человеком	Не читается человеком
pd.read_json()	pd.read_parquet()

Parquet — бинарный сжатый формат, быстрее работает с большими данными. JSON — текстовый, удобен для обмена данными, но медленнее.

7. Как проверить типы данных в DataFrame после загрузки?

Используйте метод .info() для полной информации или .dtypes для просмотра типов каждого столбца.

8. Как определить размер DataFrame (количество строк и столбцов)?

Атрибут .shape возвращает кортеж (строки, столбцы).

9. В чем разница между .loc[] и .iloc[]?

.loc[]	.iloc[]
По меткам (названиям)	По числовым позициям
df.loc["a"]	df.iloc[0]
Использует индексы строк	Использует номера строк (с 0)

.loc[] использует метки индексов и столбцов, .iloc[] использует числовые позиции.

10. Как получить данные третьей строки и второго столбца с .iloc[]?

```
df.iloc[2, 1]
```

Передаём два числа: номер строки (2 — третья строка) и номер столбца (1 — второй столбец), нумерация с 0.

11. Как получить строку с индексом "Мария" из DataFrame?

Используйте .loc[] с именем индекса.

12. Чем .at[] отличается от .loc[]?

.at[]	.loc[]
Быстрее	Медленнее
Только для одного элемента	Поддерживает срезы
df.at["a", "Имя"]	df.loc["a", "Имя"]

`.at[]` работает быстрее, но предназначен только для доступа к одному элементу, а `.loc[]` поддерживает срезы и множественный доступ.

13. В каких случаях `.iat[]` работает быстрее, чем `.iloc[]`?

При доступе к одному конкретному элементу. `.iat[]` не поддерживает срезы, только одиночные значения, поэтому он работает быстрее.

14. Как выбрать все строки, где "Город" равен "Москва" или "СПб", используя `.isin()`?

```
df[df["Город"].isin(["Москва", "СПб"])]
```

Метод `.isin()` проверяет вхождение значения в список и возвращает маску True/False.

15. Как отфильтровать DataFrame, оставив только строки, где "Возраст" от 25 до 35 лет, используя `.between()`?

Метод `.between()` включает границы по умолчанию. Можно использовать `inclusive="neither"` для исключения границ.

16. В чем разница между `.query()` и `.loc[]` для фильтрации данных?

<code>.query()</code>	<code>.loc[]</code>
Использует строковые выражения	Использует булевы условия
<code>df.query("Возраст > 25")</code>	<code>df[df["Возраст"] > 25]</code>
Удобнее для сложных условий	Более явный синтаксис

`.query()` позволяет писать условия в виде строк (похоже на SQL), а `.loc[]` использует булевы массивы.

17. Как использовать переменные Python внутри `.query()`?

```
min_age = 25
df.query("Возраст > @min_age")
```

Используйте символ `@` перед именем переменной внутри строки запроса.

18. Как узнать, сколько пропущенных значений в каждом столбце DataFrame?

Метод `.isna()` создаёт булеву маску (True для NaN), а `.sum()` считает количество True в каждом столбце.

19. В чем разница между `.isna()` и `.notna()`?

<code>.isna()</code>	<code>.notna()</code>
True для NaN	True для НЕ NaN
Находит пропуски	Находит заполненные значения

`.isna()` возвращает True где значение отсутствует (NaN), `.notna()` возвращает True где значение присутствует.

20. Как вывести только строки, где нет пропущенных значений?

Метод `.dropna()` удаляет все строки, содержащие хотя бы один NaN.

21. Как добавить новый столбец "Категория" в DataFrame, заполнив его фиксированным значением "Неизвестно"?

Просто присвойте новому столбцу одно значение — оно автоматически распространится на все строки.

22. Как добавить новую строку в DataFrame, используя `.loc[]`?

```
df.loc[len(df)] = ["Антон", 32, "Москва"]
```

Используйте `.loc[]` с новым индексом (например, `len(df)` для добавления в конец) и присвойте список значений.

23. Как удалить столбец "Возраст" из DataFrame?

```
df.drop(columns=["Возраст"])
```

```
# или
```

```
del df["Возраст"]
```

Используйте `df.drop(columns=["Возраст"])` или оператор `del df["Возраст"]`.

24. Как удалить все строки, содержащие хотя бы один NaN, из DataFrame?

Метод `.dropna()` без параметров удаляет все строки, в которых есть хотя бы одно NaN.

25. Как удалить столбцы, содержащие хотя бы один NaN, из DataFrame?

Используйте `.dropna(axis=1)` — параметр `axis=1` указывает, что удалять нужно столбцы.

26. Как посчитать количество непустых значений в каждом столбце DataFrame?

Метод `.count()` считает только те значения, которые не являются NaN.

27. Чем `.value_counts()` отличается от `.nunique()`?

<code>.value_counts()</code>	<code>.nunique()</code>
Показывает частоту каждого значения	Показывает только количество уникальных
Возвращает Series с подсчётами	Возвращает число
<code>df["Город"].value_counts()</code>	<code>df["Город"].nunique()</code>

`.value_counts()` показывает сколько раз встречается каждое уникальное значение, а `.nunique()` возвращает только количество уникальных значений.

28. Как определить сколько раз встречается каждое значение в столбце "Город"?

```
df["Город"].value_counts()
```

Используйте метод `.value_counts()`, который возвращает частоту каждого уникального значения, отсортированную по убыванию.

29. Почему `display(df)` лучше, чем `print(df)`, в Jupyter Notebook?

<code>display(df)</code>	<code>print(df)</code>
HTML-рендеринг (красивая таблица)	Текстовый вывод
Поддержка прокрутки	Обрезает длинные таблицы
Можно стилизовать (<code>.style</code>)	Без форматирования

`display()` использует HTML-форматирование, делает таблицу красивой с возможностью прокрутки и стилизации, в отличие от обычного `print()`.

30. Как изменить максимальное количество строк, отображаемых в DataFrame в Jupyter Notebook?

```
pd.set_option("display.max_rows", 100)
```

Используйте `pd.set_option("display.max_rows", N)`, где `N` — максимальное количество строк для отображения. Для отключения лимита укажите `None`.